

tmux documentation

§01 — Termux Environment Overview

Revision: 1 **Last modified:** 2026-05-22T07:20:02Z **Authority:** vasic-digital tmux project (research-only) **Maintainer:** milosvasic **Scope:** What Termux is and the Android userland constraints relevant to porting the tmx wrapper

1. What Termux is

Termux is an Android terminal-emulator + Linux-userspace add-on app. It does NOT virtualise Linux (no chroot, no namespaces, no PRoot in the default install) — instead it ships a packaged collection of Linux userland binaries (bash, coreutils, tmux, openssh, ...) compiled against **Android's own bionic libc**, installed inside the Termux app's private data directory.

Architecturally Termux is "Linux userland running as a regular unprivileged Android app." Every Termux process is a child of the Android app_process for com.termux, runs as the same Linux UID Android assigns the Termux app, inherits its app sandbox, and sees the same /proc view any other unprivileged Android app sees.

The project is hosted at <https://github.com/termux/termux-app> (the app) and <https://github.com/termux/termux-packages> (the package recipes).

2. Filesystem layout

The Termux app's private data directory is /data/data/com.termux/. Two well-known children are exported as environment variables:

Env	Path	Linux analogue
\$PREFIX (deprecated name \$TERMUX__PREFIX)	/data/data/com.termux/files/usr	/usr
\$HOME (also \$TERMUX__HOME)	/data/data/com.termux/files/home	/home/ \$USER
\$TERMUX__ROOTFS	/data/data/com.termux/files	(no exact analogue)

Source: <https://github.com/termux/termux-packages/wiki/Termux-file-system-layout> and <https://github.com/termux/termux-packages/wiki/Termux-execution-environment>.

The Android security model prohibits Termux from writing to /bin, /etc, /usr, /var, etc. at the actual filesystem root — these are part of the read-only /system partition and belong to Android itself. Every Linux-userland convention you'd expect at /etc lives under \$PREFIX/etc, /var under \$PREFIX/var, /tmp under \$PREFIX/tmp. Programs that hardcode absolute Linux paths require porting patches per the [Common porting problems wiki](#).

The \$PREFIX paths are **baked into binaries at compile time** — if you copy a Termux binary to a phone whose Termux app is installed under a different package name (e.g. forked builds like

Termux:Float), the binary will not find its libraries or config. There is no \$PREFIX relocation at runtime.

3. libc: bionic, not glibc

Termux binaries link against **Android's bionic libc**, not glibc. Bionic is Google's minimalist BSD-licensed C library, written specifically for embedded / mobile use.

Practical consequences for our port:

- **Missing or non-conformant POSIX/GNU APIs.** The bionic status doc enumerates which standard APIs are available at which Android API level — see <https://android.googlesource.com/platform/bionic/+HEAD/docs/status.md>. Notable: `pthread_cancel` is missing and explicitly flagged as "unlikely to ever be implemented."
- **DNS resolver path.** Bionic does NOT use `/etc/resolv.conf`; DNS goes through the Android system service. A pure-Go binary built with `CGO_ENABLED=0` will hit hard-coded `[ : : 1 ] : 53` and fail. Detail in §02.
- `tmpnam()` / `TMPDIR`. Older Bionic ignored `$TMPDIR`; fixed in Bionic commit 439ebbd3 but worth knowing about for legacy devices. (Source: Termux common-porting-problems wiki.)
- `setuid()` **blocked since Android 9.** Seccomp filters at the zygote level block `setuid()` and friends for app processes. Termux is single-user by design — there is nothing to give up. (Source: Termux common-porting-problems wiki.)

For our purposes (tmux + a Go state daemon + shell scripts) bionic is sufficient — tmux already builds cleanly under bionic (Termux ships it as a maintained package).

4. What is NOT available

Linux concept	Termux availability	Note
<code>systemd / systemctl</code>	absent	Android doesn't use systemd. No <code>--user</code> instance. https://github.com/termux/termux-packages/discussions/17585
<code>init.d / rc.d</code>	absent	App-lifecycle-driven; use the Termux:Boot add-on app.
<code>cgroups v1 / v2</code> writable from userspace	absent for unprivileged user	Kernel may have <code>cgroup-v2</code> but the Termux UID has no write access to <code>cgroup.procs</code> . https://dev.to/elenbit/unreliable-linux-containers-on-android-addressing-integration-networking-and-stability-for-2p5b
<code>setuid()</code> / <code>setuid</code> binaries	blocked since Android 9	seccomp policy on zygote-spawned app processes.
<code>journalctl</code>	absent	No systemd → no journal. Use <code>logcat</code> or Termux's own <code>stderr</code> .
D-Bus user session	absent in default install	Some apps install <code>dbus</code> , but no auto-start.
<code>/proc</code> writable for OOM tuning	partial	<code>/proc/\$pid/oom_score_adj</code> is writable but <code>lmkd</code> may override (see §03).
<code>udev / device hotplug</code>	absent	Android handles device events itself.
<code>/etc/passwd / /etc/shadow</code>	absent	Termux is single-user; identity comes from Android UID.

Linux concept	Termux availability	Note
Privileged ports (<1024)	unavailable	<code>bind()</code> returns <code>EACCES</code> for the unprivileged app.

5. SELinux on Android

Every Android process runs under an SELinux context. Termux processes inherit `u:r:untrusted_app_xx:s0:...` (the exact context depends on Android version + target SDK). Key consequences:

- **App-data-exec restriction (Android 10+).** Per the Termux execution-environment wiki: apps targeting `targetSdkVersion ≥ 29` *cannot* exec arbitrary files from `/data/data/<package>` — Google's W^X mitigation. Termux works around this by targeting an older SDK + a custom dynamic linker bridge (`libtermux-exec.so`, preloaded via `$LD_PRELOAD`). Operators who sideload a Termux binary outside Termux's own preload setup hit `Permission denied` (logged as `avc: denied` in `logcat`). Source: <https://github.com/termux/termux-packages/wiki/Termux-execution-environment>.
- `/system/bin` **exec is allowed** under the `app_data_file` → `system_file` transition, so calling out to Android system binaries (`getprop`, `am`, `pm`, `dumpsys`, `ip`) works.
- **No mount / chroot / unshare capability.** Container-style isolation needs root (Magisk / KernelSU).

6. Default \$PATH and shell

From the Termux execution-environment wiki:

- **Android ≥ 7 (Nougat, API 24):** `PATH=/data/data/com.termux/files/usr/bin`; `$LD_PRELOAD=/data/data/com.termux/files/usr/lib/libtermux-exec.so`.
- **Android < 7:** `PATH=/data/data/com.termux/files/usr/bin:/data/data/com.termux/files/usr/bin/applets`; `$LD_LIBRARY_PATH=/data/data/com.termux/files/usr/lib`.

The default login shell is `bash` (Termux ships `bash` as `$PREFIX/bin/bash`). `zsh` is available via `pkg install zsh`. **Bash 5.x** — not the macOS-default Bash 3.2 — so the `tmx-shell-init.sh` POSIX-sh discipline gives us Termux for free.

7. Why this matters for the tmx port

- Filesystem paths in `tmx` (the wrapper, the Go state daemon, the SSH dispatcher) **MUST** resolve at runtime via `$PREFIX / $HOME`, not via hardcoded `/usr/local/bin/...`. Our existing wrapper already does this (`TMX_DIR="$(cd "$(dirname "$0")" && pwd)"`) — good.
- We cannot rely on `systemd / cgroups / setuid / mount` namespaces for isolation — those gates simply do not exist for an unprivileged Termux process. The substitute is POSIX `setrlimit` + Android's own `lmkd` OOM killer. Details in §03.
- The shell-init prompt + `[ -t 0 ]` guard from the v1.0.9 spec works as-is on Termux: `stdin` and `stdout` are TTYs when the operator opens the Termux terminal window; they are not TTYs when an `Termux:Boot` script or `sshd` runs the script non-interactively.

- The `command= directive + SSH_ORIGINAL_COMMAND` machinery in `tmx-ssh-dispatch.sh` should work because Termux's OpenSSH is a standard build with only Termux-prefix path changes. Verified in §04.

Sources

- <https://github.com/termux/termux-packages/wiki/Termux-file-system-layout>
- <https://github.com/termux/termux-packages/wiki/Termux-execution-environment>
- <https://github.com/termux/termux-packages/wiki/Common-porting-problems>
- <https://android.googlesource.com/platform/bionic/+/HEAD/docs/status.md>
- <https://github.com/termux/termux-packages/discussions/17585> (systemctl failure discussion)
- <https://dev.to/elenbit/unreliable-linux-containers-on-android-addressing-integration-networking-and-stability-for-2p5b> (cgroup unavailability)
- <https://lwn.net/Articles/936953/> (LWN's overview of Termux internals)